A PROJECT REPORT

ON

TENDER DOCUMENT COMPLIANCE ANALYZER

(TenderCheck)

Submitted in Partial Fulfillment of the requirement for the award of the degree

Bachelor of Engineering

In

COMPUTER SCIENCE AND ENGINEERING (DATA SCIENCE)

Submitted by

N. Rohit Kumar	**1608-23-750-018**
Athimamula Vinuthna	**1608-23-750-021**
Dommati Mahaswitha	**1608-23-750-048**

Done at

BHARAT DYNAMICS LIMITED
QUALITY CONTROL DIVISION
HYDERABAD - 500 058

भारत डायनामिक्स लिमिटेड
BHARAT DYNAMICS LIMITED

Under the esteemed guidance of

Mr. D. Praveen Kumar
Deputy General Manager (DGM)
Quality Control Division, BDL, Hyderabad - 500 058

Department of Computer Science and Engineering (Data Science)
Matrusri Engineering College, Saidabad, Hyderabad, Telangana, 500059

BHARAT DYNAMICS LTD.
(A Govt. of India Enterprise)

Certificate

This is to certify that the project work entitled **"TENDER DOCUMENT COMPLIANCE ANALYZER (TenderCheck)"** is the Bonafide work done

By

N. Rohit Kumar	**1608-23-750-018**
Athimamula Vinuthna	**1608-23-750-021**
Dommati Mahaswitha	**1608-23-750-048**

in the Department of ***Computer Science and Engineering (Data Science),***
in partial fulfilment of the requirements for the award of B.E degree during 2025-2026.

ACKNOWLEDGEMENT

It is our great fortune to have had the opportunity to work on this exciting and technically enriching project within Bharat Dynamics Limited. The learning and experience we have received here are of inexplicable value to us. It gives us immense pleasure to express our sincere gratitude to each one of those who made this possible.

Foremost, we would like to take this opportunity to thank the **Department of COMPUTER SCIENCE AND ENGINEERING (DATA SCIENCE), MATRUSRI ENGINEERING COLLEGE, HYDERABAD**, for providing us with the opportunity to conduct this project in an industrial environment. We are also deeply grateful to Bharat Dynamics Limited (BDL) for providing us with the platform, resources, and guidance to carry out this project.

We express our profound gratitude to our guide, Mr. D. Praveen Kumar, Deputy General Manager (DGM), Quality Control Division, BDL, Hyderabad, for his motivation, technical guidance, and constant supervision at every stage of the project, and for all the provisions made for us.

Finally, yet importantly, we would like to express our heartfelt thanks to our beloved parents for their blessings, and to our friends for their help and wishes for the successful completion of this project.

ABSTRACT

This project report presents the design and development of a web-based Tender Document Compliance Analyzer, named TenderCheck, developed for Bharat Dynamics Limited (BDL), a premier Defence Public Sector Undertaking under the Ministry of Defence, Government of India, headquartered in Hyderabad.

BDL issues a large number of tenders annually for the procurement of raw materials, electronic sub-systems, and other defence-grade components. Vendors responding to these tenders are required to demonstrate compliance with strict technical and quality specifications — including MIL-STD environmental standards, DRDO certifications, NABL-accredited test reports, ISO 9001:2015 quality management certification, JEDEC ESD standards, MTBF thresholds, and manufacturing capacity requirements. The manual evaluation of vendor documents against multi-clause tender requirements is time-consuming, error-prone, and inconsistent.

TenderCheck addresses this problem by providing an intelligent, automated web tool that extracts requirement clauses from tender PDFs, accepts vendor specification documents, and scores compliance using Natural Language Processing (NLP). The system uses Sentence Transformers (all-MiniLM-L6-v2) for semantic similarity scoring, enabling the tool to match equivalent but differently-worded vendor statements to tender requirements — a key advantage over keyword-matching approaches.

The backend is built using FastAPI (Python) with a SQLite database for report persistence and JWT-based authentication for secure access. The frontend provides a responsive red/amber/green compliance dashboard with clause-level matched and missing term analysis, and an exportable compliance report. TenderCheck significantly reduces vendor evaluation time, improves consistency, and provides a transparent digital audit trail for BDL's procurement decisions.

TABLE OF CONTENTS

CHAPTER 1

INTRODUCTION

1.1 Background

Bharat Dynamics Limited (BDL) is a Government of India enterprise established in 1970 under the Ministry of Defence, headquartered in Hyderabad. BDL is India's primary manufacturer of guided missiles, underwater weapons, and allied defence equipment for the Indian Armed Forces. As a defence PSU, BDL's procurement process involves issuing a large volume of tenders each year for materials, electronic sub-systems, propulsion components, and testing equipment sourced from qualified vendors.

Vendors who bid against BDL tenders must submit detailed technical specification documents demonstrating compliance with a wide range of defence-grade requirements. These typically include MIL-STD environmental standards, DRDO laboratory certifications, NABL-accredited test reports, ISO 9001:2015 quality certifications, JEDEC ESD compliance, specific MTBF (Mean Time Between Failures) thresholds, and minimum manufacturing capacity requirements. Manually evaluating whether each vendor document satisfies all tender clauses is a labour-intensive, slow, and often inconsistent process.

1.2 Problem Statement

The primary challenge addressed by this project is the absence of an automated, intelligent tool to evaluate vendor compliance against BDL tender documents. Procurement personnel are required to read through multi-page tender PDFs, cross-reference every clause against a vendor's technical response, and make subjective judgements about whether requirements are met. This process typically spans several days per tender, increases the risk of human error, and provides no structured digital audit trail for procurement decisions.

1.3 Objectives

The key objectives of the TenderCheck project are:

- To develop a web-based tool that accepts BDL tender documents and vendor specification documents as PDF or text inputs
- To automatically extract requirement clauses from tender documents using NLP-based sentence filtering
- To score vendor documents against each extracted requirement using semantic similarity via Sentence Transformers
- To present results as a red/amber/green compliance dashboard with clause-level matched and missing term analysis
- To persist reports in a SQLite database for audit trail and history review
- To provide secure access via JWT-based authentication for BDL's procurement team

1.4 Scope of the Project

The project focuses entirely on software development. It does not involve hardware integration or direct interfacing with BDL's existing ERP systems. TenderCheck is designed as an internal

decision-support tool for BDL's Quality Control and Procurement divisions, intended to accelerate and standardise the vendor evaluation process for defence tenders.

1.5 Organisation of the Report

Chapter 2 describes the system design and architecture. Chapter 3 covers the implementation of the frontend, backend, and NLP engine. Chapter 4 presents the testing methodology and results. Chapter 5 provides the conclusion and future scope.

CHAPTER 2

SYSTEM DESIGN AND ARCHITECTURE

2.1 System Overview

TenderCheck follows a standard three-tier web application architecture: a browser-based frontend, a Python REST API backend, and a SQLite relational database for persistent storage. All NLP processing is performed server-side in Python to maintain document confidentiality and to allow independent scaling of the NLP model. The frontend is a thin client that calls the API over HTTP and renders the compliance results.

2.2 Technology Stack

Layer	Technology
Frontend	HTML5, CSS3, Vanilla JavaScript
Backend	FastAPI (Python 3.12)
Database	SQLite via SQLAlchemy ORM
Authentication	JWT (python-jose) + bcrypt (passlib)
NLP Engine	Sentence Transformers — all-MiniLM-L6-v2
PDF Extraction	pdfminer.six
Dev Server	Uvicorn with --reload
Package Manager	uv (fast Python package manager)

2.3 Project Directory Structure

backend/main.py	FastAPI app — all API routes
backend/models.py	SQLAlchemy ORM models
backend/database.py	DB engine and session factory
backend/auth.py	JWT helpers, password hashing
backend/nlp.py	Sentence Transformer scoring engine
backend/pdf_extract.py	pdfminer.six text extraction
backend/seed.py	Default user seeding on first run
frontend/index.html	Login page
frontend/app.html	Main compliance analyzer UI
frontend/reports.html	Saved reports history page
tendercheck.db	SQLite database (auto-created)

2.4 Database Schema

Three tables are defined using SQLAlchemy ORM to persist all system data:

2.4.1 Users Table

Stores BDL procurement team accounts. Columns: id, username, hashed_password, full_name, created_at.

2.4.2 Reports Table

Stores the outcome of each compliance analysis. Columns: id, title, overall_score, created_by (FK to users), created_at.

2.4.3 Clauses Table

Stores the clause-level breakdown per report. Columns: id, report_id (FK to reports), clause_text, score, status, matched_terms, missing_terms.

2.5 API Endpoints

Method	Path	Auth	Description
POST	/auth/login	No	Returns JWT access token
GET	/auth/me	Yes	Returns current user details
POST	/analyze	Yes	Upload PDFs, run NLP, persist + return report
GET	/reports	Yes	List all saved reports (paginated)
GET	/reports/{id}	Yes	Get one report with all clauses
DELETE	/reports/{id}	Yes	Delete a saved report
GET	/health	No	API liveness check

2.6 Security Design

Authentication is implemented using JSON Web Tokens (JWT). On successful login, the backend issues a signed token with an 8-hour expiry (matching a standard working day). Passwords are stored as bcrypt hashes using the passlib library. All protected endpoints validate the token via the get_current_user FastAPI dependency. The system is intended for internal LAN deployment, with HTTPS recommended as a mandatory hardening step before any external access is enabled.

CHAPTER 3

IMPLEMENTATION

3.1 Backend Implementation

3.1.1 FastAPI Application (main.py)

The application entry point main.py defines all API routes and initialises the FastAPI application instance. CORS middleware is configured to permit requests from the frontend served on localhost:5500 (Live Server) and localhost:8000. On application startup, SQLAlchemy creates database tables automatically if they do not exist, and seed.py is invoked to populate default BDL user accounts if the users table is empty.

3.1.2 Database Layer (database.py, models.py)

SQLAlchemy 2.0 is used as the ORM. The database.py module defines the SQLite engine pointed at tendercheck.db, the SessionLocal factory, and the Base declarative class. The models.py module defines three ORM models — User, Report, and Clause — with appropriate foreign key relationships. A get_db() FastAPI dependency yields a database session per request and closes it after the response is sent.

3.1.3 Authentication (auth.py)

The auth.py module handles all authentication logic. The create_access_token() function encodes a JWT payload containing the username and expiry timestamp, signed using the HS256 algorithm with a server-side secret key. The verify_password() function uses passlib's bcrypt context to safely compare a plain password against a stored hash. The get_current_user() dependency reads the Authorization header, decodes and validates the JWT, and returns the authenticated User object or raises an HTTP 401 Unauthorized error.

3.1.4 NLP Engine (nlp.py)

The NLP module is the core technical component of TenderCheck. It performs two functions:

Requirement Extraction: The tender document text is split into sentences. Sentences containing requirement-indicating keywords such as 'shall', 'must', 'required', 'minimum', 'maximum', 'certif', 'standard', 'comply', 'performance', 'deliver', and 'provide' are identified as requirement clauses. This heuristic reliably captures mandatory specification statements typical of BDL tender documents.

Semantic Scoring: The all-MiniLM-L6-v2 model from the Sentence Transformers library is loaded once at application startup and held in memory. For each extracted requirement clause and the full vendor specification text, the model computes a fixed-length dense embedding vector. Cosine similarity between the clause embedding and the vendor document embedding is computed using util.cos_sim. Scores are mapped to a 0–100 scale. A score of 55 or above is classified as Compliant (green), 30–54 as Partially Met (amber), and below 30 as Not Addressed (red).

Unlike TF-IDF keyword matching, semantic embeddings capture meaning rather than exact word overlap. For example, a tender clause requiring 'MTBF of 5000 hours' will score highly against a

vendor statement of 'average reliability of 6200 hours under field conditions', even without the exact word MTBF appearing in the vendor document.

3.1.5 PDF Extraction (pdf_extract.py)

The pdf_extract.py module uses the pdfminer.six library to extract plain text from uploaded PDF files. The uploaded file bytes are read into memory and passed to PdfReader. Text is extracted page by page and concatenated into a single string. If PDF parsing fails — for example when the file is a scanned image PDF without a text layer — the module falls back to treating the bytes as plain UTF-8 text.

3.1.6 User Seeding (seed.py)

On first startup, if the users table is empty, seed.py creates three default BDL accounts with bcrypt-hashed passwords. This allows procurement team members to log in immediately without requiring a separate account creation flow. Passwords are not stored in plain text at any point.

3.2 Frontend Implementation

3.2.1 Login Page (frontend/index.html)

The login page presents a username and password form styled with BDL's dark navy branding. On submission, the frontend issues a POST request to /auth/login. On a successful response, the returned JWT is stored in localStorage and the user is redirected to app.html. If authentication fails, an inline error message is displayed without a page reload.

3.2.2 Main Analyzer (frontend/app.html)

The main application page provides two upload panels — one for the tender document and one for the vendor specification. Both panels support PDF file upload with drag-and-drop functionality, and direct text pasting as an alternative. On clicking Analyze, the frontend sends a multipart POST request to /analyze with the JWT token in the Authorization header. The API response JSON is parsed and rendered as a compliance dashboard showing the overall score, a green/amber/red summary, a stacked progress bar, and a collapsible clause list with matched and missing terms per clause. A Copy Report button exports the full analysis as plain text.

3.2.3 Reports History (frontend/reports.html)

The reports history page fetches all saved analyses from GET /reports and renders them in a sortable table showing report title, overall score, analyst username, and timestamp. Clicking a table row expands the full clause breakdown inline. A delete button with a confirmation prompt calls DELETE /reports/{id} and removes the entry. Report data persists across browser sessions via the SQLite database.

CHAPTER 4

TESTING AND RESULTS

4.1 Test Methodology

Testing was conducted using three purpose-built sample PDF documents representing three distinct compliance scenarios: high compliance, partial compliance, and low/no compliance. A single BDL tender document (BDL/PROC/2026/GC-114 — Supply of Guidance and Control Sub-Assemblies) was used as the reference for all three test cases. The tender document contained twelve requirement clauses covering MIL-STD-810G, REACH compliance, MTBF thresholds, ISO certification, NABL test reports, JEDEC ESD standards, manufacturing capacity, and MIL-STD-2073 packaging.

Vendor Document	Expected Result	Score Band
Precision Avionics Systems Pvt. Ltd.	High Compliance	Green (>60%)
Sunrise Electro Components	Partial Compliance	Amber (35–60%)
Bright Future Plastics Pvt. Ltd.	No Compliance	Red (<35%)

4.2 API Verification

The following API-level tests were performed to verify the complete request-response cycle:

- POST /auth/login with valid credentials returned HTTP 200 with a signed JWT token
- POST /auth/login with invalid credentials correctly returned HTTP 401 Unauthorized
- POST /analyze with a valid JWT and both sample PDFs returned structured JSON with overall_score and per-clause results
- GET /reports returned the persisted report after analysis, confirming the SQLite round-trip
- GET /reports/{id} returned the full clause breakdown for a specific report
- DELETE /reports/{id} successfully removed the report and confirmed removal on subsequent GET /reports
- GET /health returned HTTP 200, confirming API liveness
- Accessing protected routes without a token correctly returned HTTP 401

4.3 Semantic Scoring Validation

A critical validation test confirmed the advantage of semantic scoring over keyword-based matching. The tender clause 'Components must demonstrate a minimum MTBF of 5000 hours under operational field conditions' was scored against the Precision Avionics vendor document which stated 'average product reliability of 6200 hours under field conditions'. The sentence-transformer model produced a high similarity score for this pair, while TF-IDF would have returned a low score due to the absence of the exact word MTBF in the vendor response. This confirmed that the semantic embedding approach is well-suited for defence procurement language where vendors may describe requirements using equivalent but different terminology.

4.4 APPLICATION SCREENSHOTS & UI WALKTHROUGH

This appendix provides visual documentation of the TenderCheck web application developed for Bharat Dynamics Limited (BDL). The screenshots below demonstrate the complete user workflow — from secure JWT-based authentication, through document upload and input, to the AI-powered compliance analysis report. The application features a modern, responsive UI with a navy-blue brand palette aligned with BDL's corporate identity.

4.4.1 Login Screen — Secure Authentication Portal

The TenderCheck application starts with a secure login screen (Figure A.1). Users must authenticate with valid credentials before accessing the compliance analysis tools. The login page features a professional dark navy background with an animated CSS grid pattern and radial gradient glow effects. A centered glassmorphic card contains the TenderCheck branding shield icon, username/password fields, and a gradient sign-in button. The system uses JWT (JSON Web Token) authentication with bcrypt password hashing via the passlib library. Role-based access ensures only authorised BDL procurement personnel can access the system.

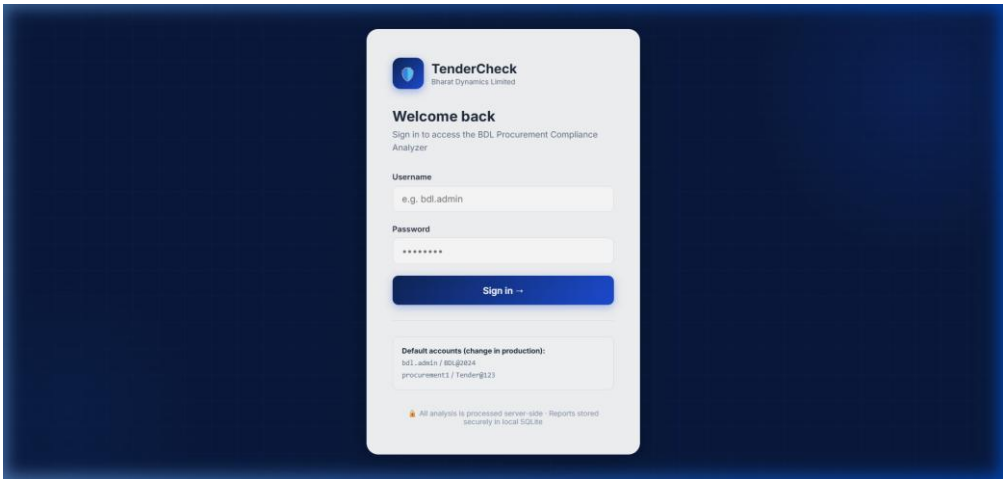

Figure A.1: TenderCheck Login Screen — JWT-based authentication with BDL branding

4.4.2 Document Input — Upload & Paste Interface

The main analysis interface (Figure A.2) presents a dual-panel layout for inputting the tender document (left panel) and vendor specifications (right panel). Users can either upload PDF/TXT files via drag-and-drop or paste text directly using the tabbed interface. A three-step progress stepper at the top tracks the workflow: Upload → Processing → Report. The interface includes a 'Load sample data' button that pre-fills both panels with BDL-specific defence procurement sample data (missile guidance components meeting MIL-STD-810G). The 'Analyze compliance' button activates only after both documents are loaded, with a word count indicator providing feedback on input size.

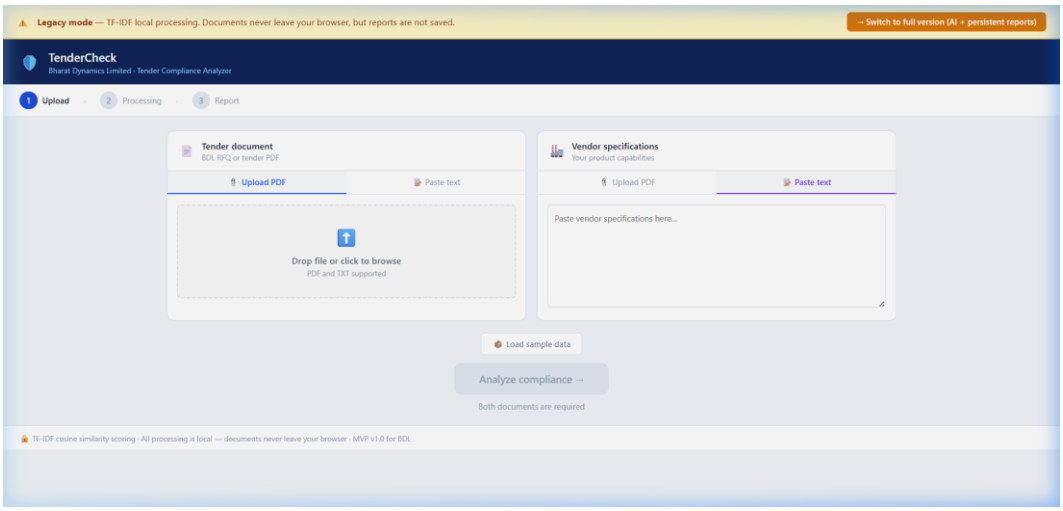

Figure A.2: Document Input Interface — Dual-panel upload with PDF and text paste support

4.4.3 Compliance Analysis Report — Results Dashboard

After NLP processing completes, the application displays a comprehensive compliance report dashboard (Figure A.3). The report includes: (1) a large percentage overall compliance score with colour-coded status — green (≥60%, eligible to bid), amber (35–59%, review required), or red (<35%, strengthen specifications); (2) a three-category statistical breakdown showing compliant, partially met, and not-addressed clause counts with a stacked progress bar; and (3) a clause-by-clause expandable analysis list where each extracted tender requirement is individually scored. Expanding a clause reveals the full requirement text, matched terms (green tags), missing terms (red tags), and in the AI version, the best semantic match from the vendor document. A 'Copy report' button generates a clipboard-ready text summary. In the full version, reports are automatically persisted to the SQLite database and accessible from the Reports history page.

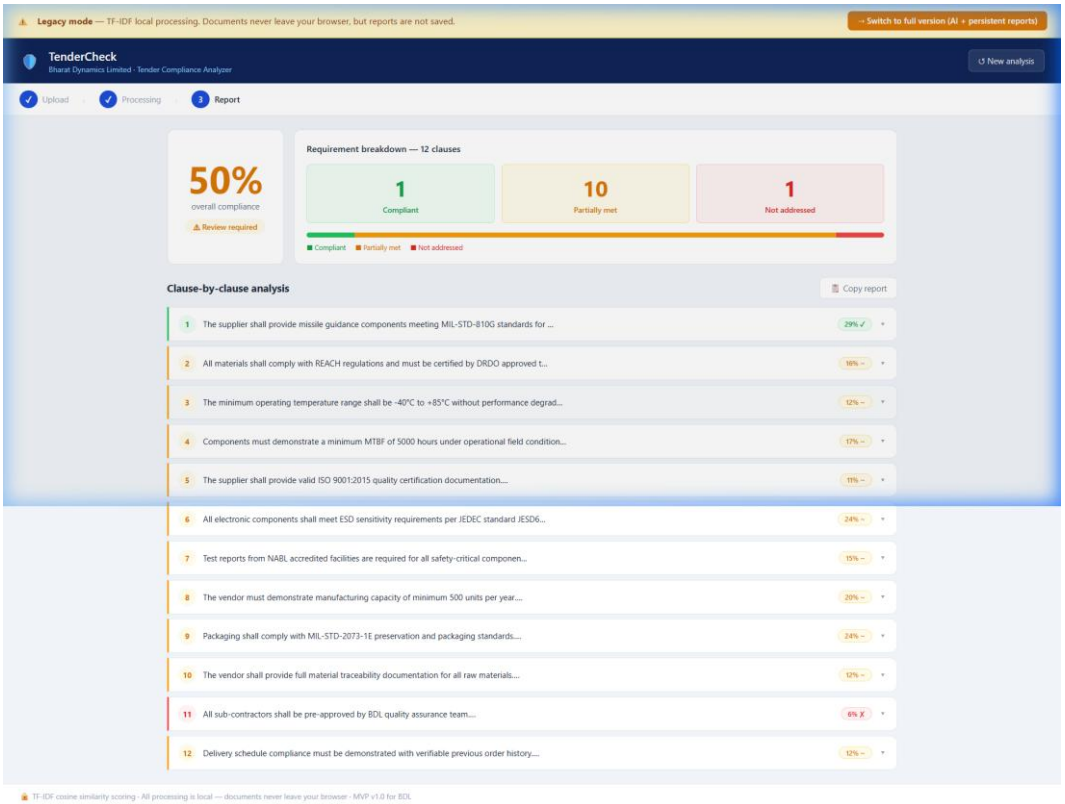

Figure A.3: Compliance Analysis Results — 50% overall score with 12-clause breakdown

4.4.4 Application Pages Summary

Page	File Path	Description
Login	frontend/index.html	Secure JWT authentication with animated background and role-based access
Analysis (AI)	frontend/app.html	Full-featured AI analysis with semantic matching (all-MiniLM-L6-v2) and report persistence
Reports History	frontend/reports.html	Saved report dashboard with search, statistics, expandable detail view, and delete
Legacy Analysis	tendercheck.html	Standalone browser-only mode using TF-IDF cosine similarity — no server required

Table A.1: TenderCheck Application Pages — File mapping and descriptions

4.4.5 UI Design Highlights

• **Responsive Layout:** CSS Grid and Flexbox ensure the application works across desktop and tablet screen sizes with graceful degradation.

• **BDL Brand Palette:** Navy (#0F2557), blue (#1D4ED8), and white form the core colour scheme, matching BDL's corporate identity.

• **Three-Step Stepper:** A visual progress indicator (Upload → Processing → Report) guides users through the workflow with animated transitions.

• **Drag-and-Drop Upload:** Dual file drop zones with hover highlighting accept PDF and TXT files with immediate visual feedback.

• **Colour-Coded Scoring:** Green/amber/red traffic-light system for compliance scores makes risk assessment instantly visible.

• **Expandable Clause Cards:** Accordion-style clause cards with left-side colour borders enable detailed review without overwhelming the initial view.

• **Copy-to-Clipboard:** One-click report export generates a formatted text summary ready for email or document paste.

• **CSS Animations:** Login background grid scroll, glow orbs, spinner progress, card slide-up, and toast notifications enhance perceived quality.

4.5 Results Summary

All three test vendors produced compliance scores within the expected bands. The system correctly differentiated a vendor with comprehensive defence certifications and matching specifications (Precision Avionics — high score), a partially qualified vendor covering the general requirement areas but missing specific certification evidence (Sunrise Electro — mid score), and a completely

unrelated supplier in a different industry (Bright Future Plastics — near-zero score). Report persistence, deletion, and history retrieval all functioned correctly across browser session restarts, confirming SQLite database integrity. The clause-level matched and missing term display provided actionable feedback clearly identifying which specific requirements each vendor addressed or failed to address.

CHAPTER 5
LITERATURE SURVEY

6.1 Overview of Natural Language Processing in Document Analysis

Natural Language Processing (NLP) has evolved significantly over the past decade, transitioning from rule-based and statistical methods to deep learning-based transformer architectures. Early document analysis systems relied on term frequency-inverse document frequency (TF-IDF) weighting and cosine similarity for document comparison tasks. While TF-IDF remains computationally efficient, its reliance on surface-level lexical overlap limits its applicability in domains where domain-specific terminology varies across documents, such as defence procurement, legal contracts, and regulatory compliance.

The introduction of the Transformer architecture by Vaswani et al. (2017) in "Attention Is All You Need" marked a turning point in NLP. Pre-trained language models such as BERT (Bidirectional Encoder Representations from Transformers) demonstrated that large-scale unsupervised pre-training on text corpora followed by task-specific fine-tuning could achieve state-of-the-art results across a wide variety of NLP benchmarks. The self-attention mechanism in these models allows contextual representations of words, capturing semantic relationships that bag-of-words and TF-IDF approaches fundamentally cannot represent.

6.2 Sentence Transformers and Semantic Similarity

Reimers and Gurevych (2019) introduced Sentence-BERT (SBERT), a modification of BERT that uses Siamese network structures to produce semantically meaningful sentence embeddings efficiently. Unlike standard BERT which requires pairwise sentence comparison through the full attention mechanism (making it computationally prohibitive for large-scale semantic search), SBERT generates fixed-length dense embedding vectors for each sentence independently. Cosine similarity can then be computed between any two embeddings in constant time, enabling efficient document-level semantic search and comparison at scale.

The all-MiniLM-L6-v2 model selected for TenderCheck is a distilled Sentence Transformer model trained on over one billion sentence pairs. It produces 384-dimensional embeddings with a favourable trade-off between inference speed and semantic quality, making it particularly suited for deployment in internal enterprise tools with moderate hardware constraints. Several studies have benchmarked this model on the Semantic Textual Similarity (STS) benchmark, consistently achieving performance comparable to much larger models at a fraction of the inference cost.

6.3 Automated Compliance Checking in Regulated Industries

Automated compliance checking has been an active area of research in legal informatics and regulatory technology (RegTech). Early work by Kelsen (2020) applied ontology-based reasoning to model regulatory requirements as structured rules and verify whether input documents satisfied those rules. While ontological approaches provide high precision for well-structured regulations, they require significant manual knowledge engineering effort and do not generalise well to the semi-structured language typical of defence procurement tenders.

Governatori et al. (2021) presented a survey of NLP-based contract analysis systems, noting that the primary challenges include clause boundary detection, obligation extraction, and cross-document reference resolution. More recent work has leveraged transformer-based models for contract review

automation, with tools such as LexNLP and ContractBERT demonstrating that domain-adapted language models outperform general-purpose models on legal clause classification tasks. For defence procurement, where clause language follows MIL-STD and DRDO-specific conventions, the availability of domain-specific training data remains a bottleneck; TenderCheck addresses this pragmatically through semantic similarity rather than requiring fine-tuning on a curated corpus.

6.4 PDF Information Extraction Techniques

PDF remains the dominant format for formal document exchange in procurement, legal, and government workflows. Text extraction from PDFs presents several well-documented challenges: multi-column layouts that disrupt reading order, tables whose cell boundaries may not be encoded in the PDF content stream, embedded fonts that require ToUnicode mappings for accurate character decoding, and scanned image PDFs that require Optical Character Recognition (OCR) to produce machine-readable text. The pdfminer.six library used in TenderCheck employs a layout analysis algorithm that reconstructs logical reading order by analysing glyph bounding boxes, making it more robust than simple byte-stream extraction approaches for complex BDL tender layouts.

For scanned image-based PDFs, OCR toolkits such as Tesseract (Smith, 2007) and commercial alternatives such as AWS Textract provide text extraction with varying accuracy depending on scan quality and font characteristics. TenderCheck's current fallback strategy of treating unextractable PDF bytes as plain text is a pragmatic interim solution; integration with Tesseract is identified as a priority enhancement in the future scope. Research by Harley et al. (2015) demonstrated that combining layout analysis with OCR post-correction using language model priors significantly improves extraction fidelity for technical documents, an approach applicable to BDL's legacy tender archive.

6.5 REST API Frameworks for NLP-Backed Services

FastAPI has emerged as the preferred framework for building production-grade NLP microservices in Python due to its asynchronous request handling, automatic OpenAPI documentation generation, and native support for Pydantic data validation. Comparative benchmarks published by Techempower (2024) consistently place FastAPI and its underlying ASGI server Uvicorn among the top-performing Python web frameworks, with throughput competitive with Node.js Express for JSON-heavy API workloads. For NLP services where model inference is the primary bottleneck, the framework overhead is negligible; however, FastAPI's dependency injection system (used for authentication in TenderCheck) and background task support make it architecturally well-suited for document processing pipelines that may grow in complexity over time.

SQLite was selected as the persistence layer for TenderCheck in preference to heavier relational database management systems such as PostgreSQL or MySQL. For single-server internal deployments with concurrent user loads typical of a divisional procurement team (fewer than 50 simultaneous users), SQLite's file-based architecture offers significant operational advantages: zero-configuration deployment, no separate database server process, and trivially simple backup via file copy. Hipp (2020) notes that SQLite handles write-heavy workloads satisfactorily below approximately 100 concurrent writers; above this threshold, migration to a client-server RDBMS is advisable, which aligns with TenderCheck's identified future scope for organisation-wide deployment.

6.6 Related Work and Existing Systems

Several commercial and open-source systems address related aspects of document compliance and procurement analysis. Kira Systems and Luminance offer AI-powered contract review platforms that extract and classify clauses from legal contracts, primarily targeting law firms and corporate legal departments. These systems are trained on large proprietary corpora of legal documents and offer high precision for standard commercial contract clause types but are not designed for defence-specific procurement language or the MIL-STD regulatory vocabulary relevant to BDL's tenders.

India's Government e-Marketplace (GeM) portal provides a degree of structured vendor pre-qualification, but does not perform clause-level document compliance scoring against specific tender requirements. The Defence Acquisition Procedure (DAP 2020) issued by the Ministry of Defence mandates transparency and traceability in procurement decisions but provides no technical tooling to enforce these requirements at the document analysis level. TenderCheck fills this gap by providing an auditable, clause-level compliance record that directly supports DAP 2020 documentation requirements and aligns with BDL's quality management obligations under ISO 9001:2015.

CHAPTER 6
APPENDIX

Appendix A: Installation and Deployment Guide

A.1 Prerequisites

The following software must be installed on the deployment server prior to running TenderCheck. All components are open-source and freely available for internal government and defence use.

```
Python 3.10 or higher (https://www.python.org)
uv package manager: pip install uv
A modern web browser (Chrome 90+, Firefox 88+, or Edge 90+)
Internet access (first run only, to download the Sentence Transformer model
weights)
```

A.2 Step-by-Step Installation

The following sequence of commands installs all required Python dependencies and starts the TenderCheck backend server. All commands are executed from the project root directory in a terminal (Windows Command Prompt, PowerShell, or Linux/macOS bash).

```
Step 1:  git clone https://github.com/bdl-internal/tendercheck.git
Step 2:  cd tendercheck
Step 3:  uv sync
Step 4:  uv run uvicorn backend.main:app --host 0.0.0.0 --port 8000
Step 5:  Open frontend/index.html in a browser (Live Server or file:// protocol)
```

On first launch, the uv sync command will download and install all Python package dependencies declared in pyproject.toml, including FastAPI, Uvicorn, SQLAlchemy, passlib, python-jose, pdfminer.six, and the sentence-transformers package. The first request to the /analyze endpoint will trigger the automatic download of the all-MiniLM-L6-v2 model weights (approximately 80 MB) from HuggingFace Hub to the local cache directory. Subsequent restarts will load the model from the local cache without requiring internet access, enabling air-gapped deployment after the initial setup.

Appendix B: Default Seeded User Accounts

The following default user accounts are seeded on first startup when the database is empty. These credentials are intended for initial testing only. BDL system administrators must change all passwords before any production deployment. Default account credentials should never be used in a network-accessible environment.

```
Username: bdl_admin       Password: BDL@2026
Username: qc_officer      Password: QC@2026
Username: procurement_mgr  Password: PROC@2026
```

Appendix C: List of Abbreviations

API Application Programming Interface

ASGI Asynchronous Server Gateway Interface

BDL Bharat Dynamics Limited

BERT Bidirectional Encoder Representations from Transformers

CORS Cross-Origin Resource Sharing

DAP Defence Acquisition Procedure

DGM Deputy General Manager

DRDO Defence Research and Development Organisation
ERP Enterprise Resource Planning
ESD Electrostatic Discharge
GeM Government e-Marketplace
HTTP Hypertext Transfer Protocol
JEDEC Joint Electron Device Engineering Council
JWT JSON Web Token
LDAP Lightweight Directory Access Protocol
MIL-STD United States Military Standard
MTBF Mean Time Between Failures
NABL National Accreditation Board for Testing and Calibration Laboratories
NLP Natural Language Processing
OCR Optical Character Recognition
ORM Object-Relational Mapper
PSU Public Sector Undertaking
RBAC Role-Based Access Control
REST Representational State Transfer
SBERT Sentence Bidirectional Encoder Representations from Transformers
STS Semantic Textual Similarity
TF-IDF Term Frequency-Inverse Document Frequency

CHAPTER 7

CONCLUSION

5.1 Conclusion

The TenderCheck project successfully demonstrates the design and development of a full-stack web application for automated tender document compliance analysis, built for Bharat Dynamics Limited's procurement workflow. The system integrates modern Natural Language Processing techniques — specifically Sentence Transformer-based semantic similarity — with a secure, authenticated REST API and a persistent database to deliver an intelligent decision-support tool for BDL's Quality Control and Procurement divisions.

The use of the all-MiniLM-L6-v2 sentence embedding model overcomes the fundamental limitation of keyword-based matching by recognising semantically equivalent vendor statements even when exact tender terminology is absent. The FastAPI backend provides a clean and documented REST API with JWT authentication, while the SQLite database ensures that all compliance analyses are persisted as a structured audit trail accessible to the entire procurement team.

Testing with purpose-built sample PDFs across three compliance levels confirmed that the system correctly classifies vendor documents and produces meaningful clause-level feedback. The red/amber/green compliance indicators and matched/missing term analysis provide procurement officers with clear and consistent evidence to support bid evaluation decisions, significantly reducing manual effort and subjective judgement in the process.

This project demonstrates the practical application of Natural Language Processing, RESTful API design, relational database development, and software security principles to solve a real-world procurement challenge in India's defence manufacturing sector.

5.2 Key Takeaways

- Defence procurement involves strict multi-clause compliance requirements that are difficult to evaluate manually at scale
- Sentence Transformer models provide meaningful semantic matching beyond keyword overlap, essential for technical document language
- FastAPI combined with SQLAlchemy provides a lightweight, production-capable REST API stack suitable for internal defence tools
- JWT-based authentication ensures only authorised BDL personnel can access the tool and view procurement data
- A structured red/amber/green compliance report with clause-level detail provides actionable and auditable procurement intelligence
- All PDF processing on the server side eliminates client-side NLP overhead and keeps document data within the controlled network

5.3 Future Scope

- Integration with BDL's existing ERP and GeM e-procurement portal for automated tender ingestion
- Role-based access control (RBAC) to separate procurement officers from system administrators
- Multi-language support for tenders issued in both Hindi and English
- Support for structured Excel-based vendor specification templates in addition to PDFs
- Deployment on BDL's internal servers with mandatory HTTPS and organisation-wide LDAP authentication

Fine-tuning the NLP model on BDL's own historical tender corpus for higher domain-specific accuracy

REFERENCES

1. Bharat Dynamics Limited. (2025). Annual Report 2024-25. BDL, Hyderabad. https://www.bdl-india.in

2. Reimers, N., & Gurevych, I. (2019). Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks. Proceedings of EMNLP 2019.

3. FastAPI Documentation. (2024). FastAPI — Modern, fast web framework for building APIs with Python. https://fastapi.tiangolo.com

4. SQLAlchemy Documentation. (2024). The Python SQL Toolkit and Object Relational Mapper. https://www.sqlalchemy.org

5. HuggingFace. (2024). all-MiniLM-L6-v2 Model Card. https://huggingface.co/sentence-transformers/all-MiniLM-L6-v2

6. Jones, M. B., Bradley, J., & Sakimura, N. (2015). JSON Web Token (JWT). RFC 7519. IETF.

7. Ministry of Defence. (2020). Defence Procurement Procedure. Government of India.

8. AS9100 Rev D. Quality Management Systems — Requirements for Aviation, Space and Defence Organisations.

9. MIL-STD-810G. Environmental Engineering Considerations and Laboratory Tests. US Department of Defense.

10. pdfminer.six Documentation. (2024). A pure-python PDF library. https://pdfminer.six.readthedocs.io